Shenzhen

Fine-Tuning Qwen3.5-27B for C-to-Rust Code Translation

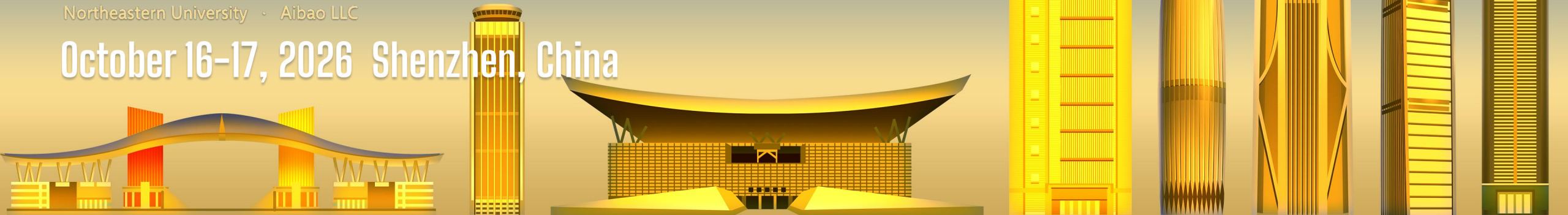
A Three-Stage Curriculum of Pretraining, Debugging-Aware SFT, and Task-Specific SFT

GOSIM Shenzhen 2026

Pu Zhao · Changdi Yang · Yixiao Chen · Yi Gao · Yifan Cao · Haochen Zeng · Yanzhi Wang

Northeastern University · Aibao LLC

October 16–17, 2026 Shenzhen, China



CONTENTS

Part 01 Background & Motivation

Part 02 Three-Stage Training Curriculum

Part 03 Evaluation with SACTOR

Part 04 Results & Takeaways



Shenzhen

01

Background & Motivation

Why C-to-Rust translation matters — and why off-the-shelf LLMs fall short



Why C-to-Rust?

C still powers most performance-critical, systems-level software — but its **manual memory management** is a persistent source of vulnerabilities:

- Buffer overflows
- Use-after-free
- Memory leaks

Rewriting these codebases in Rust eliminates entire classes of memory-safety defects **while preserving functional behavior** — but manual translation is slow, expensive, and error-prone.

Automated, LLM-assisted C-to-Rust translation is the practical path — if the model can produce safe, idiomatic Rust.

Why Rust?

Compile-time ownership & borrowing

Guarantees memory safety with no garbage collector — the whole bug class is ruled out before the program runs.

Already trusted in security-critical systems

Growing adoption in the Linux kernel, Firefox, and beyond.

The catch

Rust's strict semantics make naive, one-shot translation unreliable.



Why Not Off-the-Shelf LLMs?

Rule-based tools like **C2Rust** transliterate the C AST: the Rust compiles, but leans on raw pointers and **unsafe** blocks — low readability, little safety gain.

General-purpose LLMs translate better, yet underperform when applied directly. Three capabilities are missing:

1 **Idiomatic Rust prior**

General pretraining rarely emphasizes idiomatic patterns, standard-library usage, or ownership-aware API style.

2 **Ownership & lifetime reasoning**

Cross-language semantic equivalence is hard: the model must reason about moves, borrows, and lifetimes — not just syntax.

3 **Feedback-driven self-repair**

Real pipelines verify with compilers and tests; a usable model must interpret those signals and repair its own output.

The missing piece is specialization — so we design a three-stage fine-tuning curriculum for Qwen3.5-27B.



Shenzhen

Our Approach at a Glance

Specialize one 27B open model for C-to-Rust — three curriculum stages, one agentic benchmark

STAGE 1

Rust Continued Pretraining

1.67M examples from 7 sources — idiomatic Rust syntax & std-lib prior

→

STAGE 2

Debugging-Aware SFT

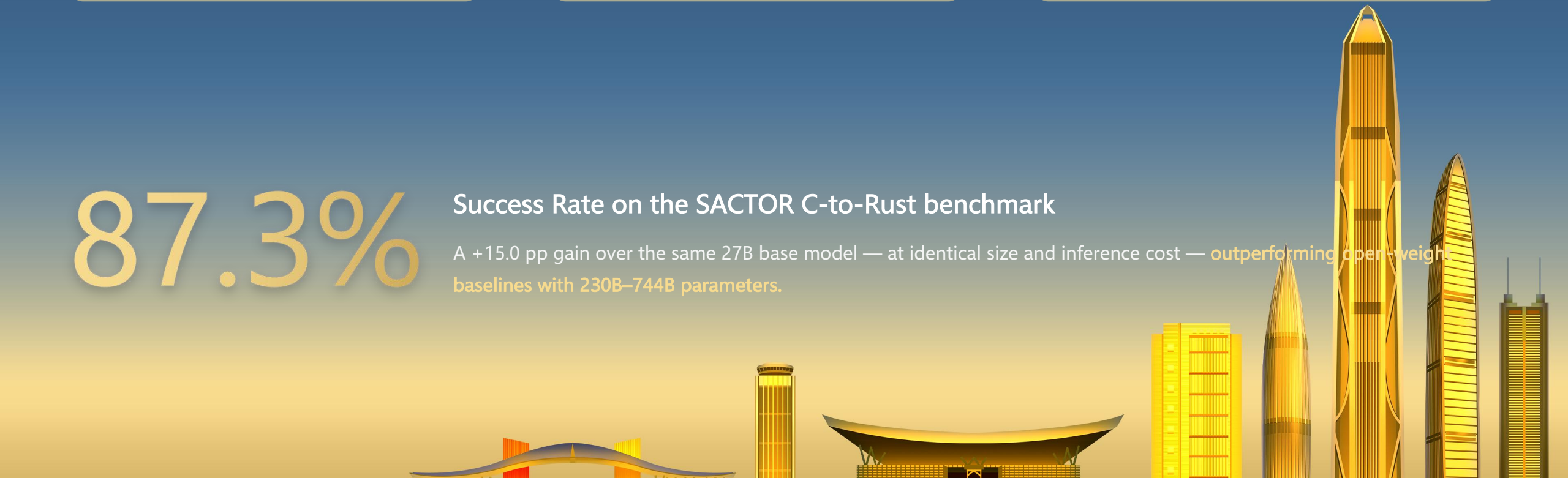
Verus data — consume verifier errors, produce minimal targeted repairs

→

STAGE 3

Task-Specific SFT

Paired C/Rust solutions — the direct source-to-target mapping



87.3%

Success Rate on the SACTOR C-to-Rust benchmark

A +15.0 pp gain over the same 27B base model — at identical size and inference cost — outperforming open-weight baselines with 230B–744B parameters.

Shenzhen

02

Three-Stage Training Curriculum

Language prior → self-repair skill → task-specific mapping



Base Model & Infrastructure

We start from **Qwen3.5-27B** (Apache 2.0) and fine-tune the released BF16 weights with **no architectural modifications**.

The unified checkpoint ships with a vision tower, but our task is purely textual — it stays loaded, **receives no gradient signal**, and plays no role in translation.

Hybrid attention

3 of 4 layers use Gated DeltaNet (linear attention, constant per-step cost); every 4th layer keeps full Gated Attention with RoPE.

Long native context

262,144 tokens — enough for a full C source file plus SACTOR's iterative compiler/test feedback in one agent turn.

27B dense · 248K vocab

Tokenizer and chat template used unmodified; no new special tokens added across all three stages.

Training setup

8× NVIDIA B300 (Blackwell Ultra), DeepSpeed ZeRO-3, BF16 full-parameter fine-tuning with LLaMA-Factory, 16,384-token sequences.

One shared recipe family across all three stages: full-parameter · BF16 · ZeRO-3 · cosine LR with 5% warmup



Stage 1 — Rust-Focused Continued Pretraining

Continue pretraining on Rust-centric text and code from **seven public Hugging Face sources** — flattened into natural continuation text (no chat templating), so the model absorbs idiomatic Rust before any task tuning.

starcoderdata (Rust subset)	380,468
Strandset-Rust-v1 (synthetic, 15 task categories)	191,008
CodeFIM-Rust-Mellum (fill-in-the-middle)	56,920
xCodeEval (synthesis + automatic program repair)	39,510

… plus three smaller sources (Magicoder OSS-Instruct Rust subset, HumanEval-Rust, a community Rust instruction dataset) for phrasing diversity.

Stage 1 recipe

Objective	continued pretraining (LLaMA-Factory pt)
Scale	1,673,289 examples · 1 epoch
LR	1.0e-6 · cosine, 5% warmup
Sequence	16,384 tokens · BF16 · ZeRO-3

FIM (prefix/middle/suffix) data strengthens reasoning over partial code — a useful prior for the repair loops used later.

Largest share is real-world GitHub Rust — naturalistic style and idiom beyond what synthetic or benchmark-derived data offers.



Stage 2 — Debugging-Aware SFT on Verus Data

microsoft/Verus Training Data (VeruSyn) pairs Rust programs whose formal proofs the Verus prover **rejects** — with the verifier error report — against **corrected versions it accepts**.

6.9M
verified programs
synthesized, prover-checked

1.2M
debugging pairs
rejected → accepted, with error report

4,557
agent CoT trajectories
multi-step repair reasoning

Why this stage helps C2Rust

The object-level task differs, but the **meta-skill transfers directly**: read a tool's structured failure output — a Verus report here, a Rust compiler error or failing E2E test in SACTOR — and produce a **minimal, targeted revision** instead of a wholesale rewrite.

Quality control: anti-cheating filters

Rejects programs that weaken the spec, sneak in **assume / admit** escape hatches, or dodge verification with non-terminating loops — without them, models learn to cheat rather than to prove.

Recipe: initialized from Stage 1 · LR 2.0e-7, cosine 5% warmup · 2 epochs · seq 16,384 · full-parameter, ZeRO-3, BF16



Stage 3 — Task-Specific SFT on C2Rust-Moxin

The final stage teaches exactly the mapping used at inference: each example pairs a **C solution as input** with a **Rust solution as the target**, drawn from LeetCode problems that have both.

functions/	One-to-one C2Rust function pairs from LeetCode / LeetCode-CN — short, self-contained, algorithmically well-specified.	TRAINED
programs/	Complete runnable CLI / small applications — closer to the SACTOR evaluation targets: must compile and run end-to-end.	TRAINED
projects/	Full multi-file codebases for large-scale migration studies — excluded from training; a deliberate scope limit.	NOT USED

Stage 3 recipe

Initialized	Stage 2 checkpoint
LR	2.0e-7 · cosine, 5% warmup
Epochs	2 · seq 16,384
Backend	full-parameter · ZeRO-3 · BF16

No project-scale exposure at training time — pointer-heavy real-world codebases remain future work.



The Curriculum, Revisited

Capabilities stack — by evaluation time, the repair loop gets exactly the skills it needs

STAGE 1

Speak idiomatic Rust

1.67M examples build the syntax / std-lib / idiom prior that one-shot generation lacks.

→

STAGE 2

Repair from feedback

1.2M debugging pairs teach minimal, targeted fixes driven by structured error reports.

→

STAGE 3

Translate C to Rust

Paired LeetCode solutions lock in the exact source→target mapping the benchmark demands.

Curriculum order matters: language first, repair second, task last — no single stage solves C2Rust alone.

Shenzhen

03

Evaluation with SACTOR

Execution-based verification: compile, run, compare, repair



The SACTOR Pipeline

We adopt the framework **unmodified**: a translation counts as successful only when the generated Rust **compiles** and **reproduces the observable behavior** of the original C program on every test.

Verification is behavioral — no reference Rust source, no textual similarity.

1. Task division

Decompose the C program into translation units (types, globals, functions); order them by dependencies.

↓

2. Unidiomatic translation

Interface-preserving Rust — raw pointers and unsafe blocks allowed to lock in behavior first.

↓

3. Idiomatic refinement

Reconstruct ownership and replace raw pointers where possible — safer, idiomatic Rust.

↓

4. Verification & repair

FFI-linked E2E tests against the original C program; failures return structured compiler/test feedback.

Fail → revise with feedback, up to 6 translation attempts per program → else counted as failure



Benchmark, Metrics & Inference Setup

200
C programs in the benchmark

~120
derived from IBM Project CodeNet

92 · argv
read input via command-line args

108 · stdin
read from standard input

Every program ships test inputs plus reference outputs captured from the **original C binary**. The input convention (argv / stdin) is told to the system explicitly, so interface mismatches never count as translation errors.

Success Rate (SR) — the primary metric

$$SR = N_{pass} / N_{total} \times 100\%$$

averaged over 5 random seeds (stochastic sampling)

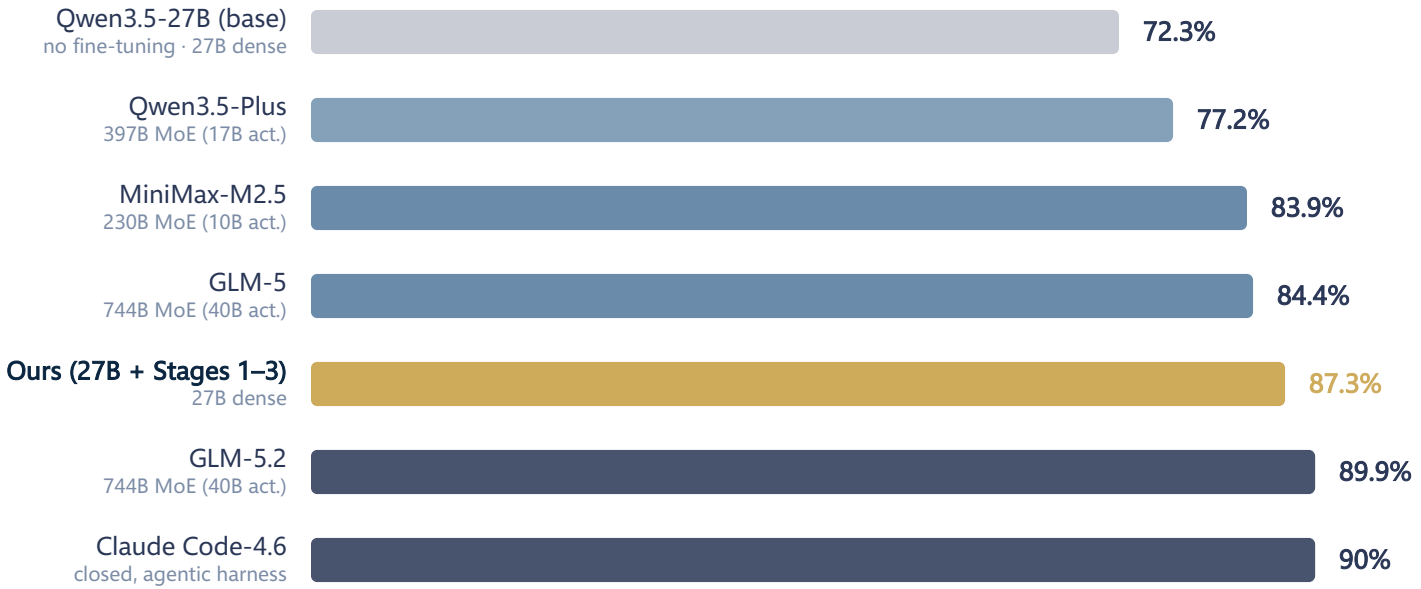
· A program passes only if the Rust compiles and all E2E tests agree with the C reference.

· No fixed Rust reference implementation — behavior, not text, is what is graded.

Inference: **temp 0.6 · top-p 0.95 · top-k 20 · ≤1,536 output tokens · ≤6 attempts** — held fixed across all models.



Main Results: a 27B Model Beats 744B-Class Baselines



+15.0 pp

over its own base checkpoint — **same 27B weights, same inference cost**. The curriculum, not scale, does the work.

Above 230B–744B open baselines (Qwen3.5-Plus, MiniMax-M2.5, GLM-5).

Trails only GLM-5.2 (89.9%) and Claude Code-4.6 (90.0%) by ≈2.7 pp.

Shenzhen

04

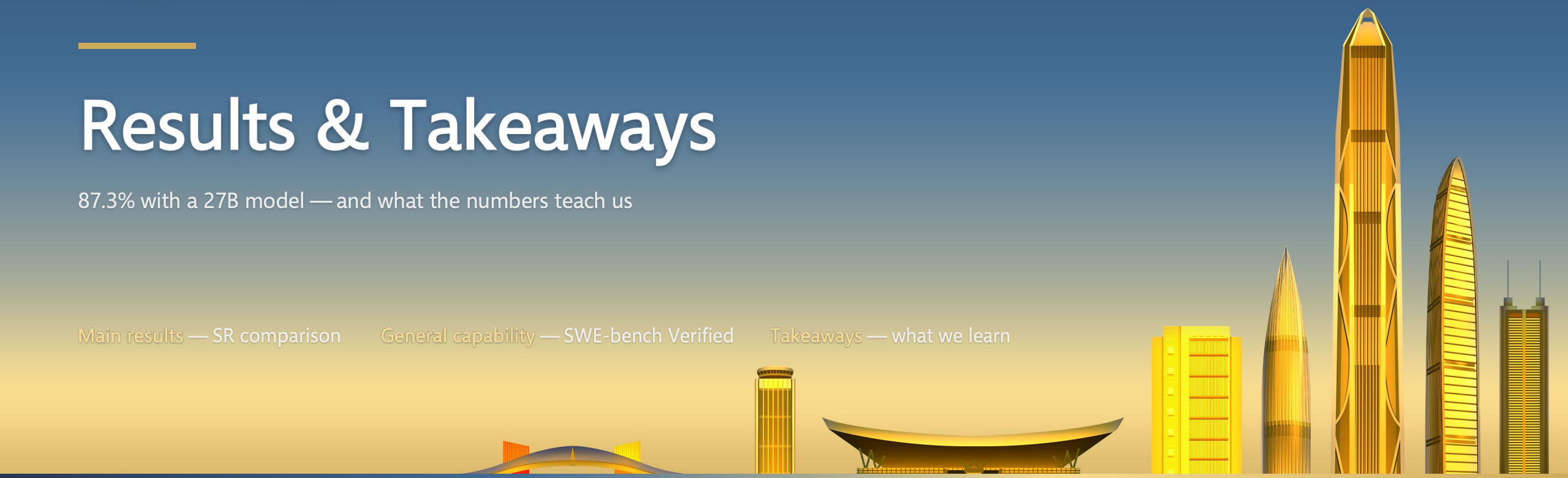
Results & Takeaways

87.3% with a 27B model — and what the numbers teach us

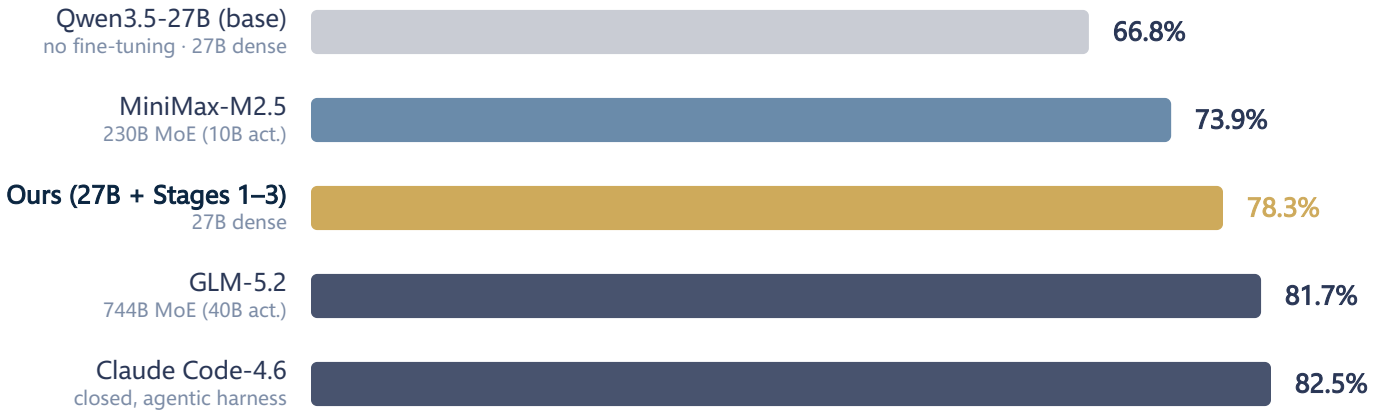
Main results — SR comparison

General capability — SWE-bench Verified

Takeaways — what we learn



Beyond Translation: Debugging Rust Code



The task

Additional evaluation beyond the paper: locate and fix bugs in Rust code sampled from a subset of **microsoft/Verus_Training_Data**.

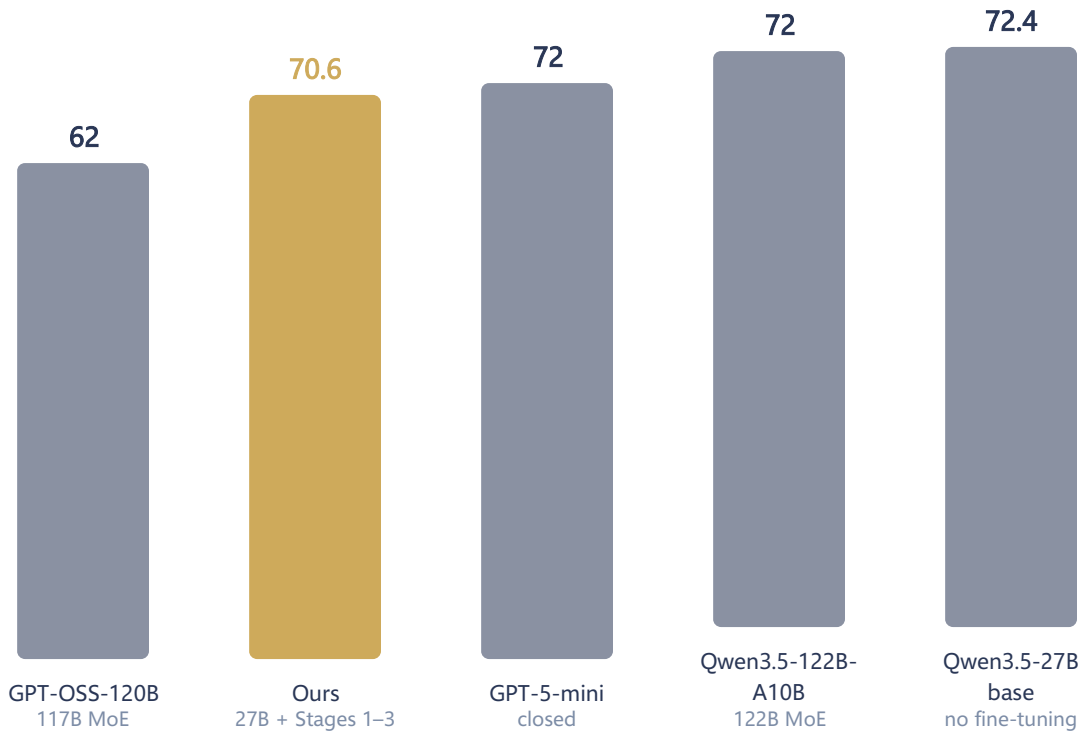
+11.5 pp

over its own base checkpoint — the same curriculum transfers to **debugging**, not just translation.

With 27B weights, it closes most of the gap to 230B–744B-class systems (MiniMax-M2.5, GLM-5.2, Claude Code-4.6).

Does Specialization Hurt General Coding?

SWE-bench Verified (pass@1): resolve real GitHub issues — **no Rust, no C2Rust emphasis**. A check on catastrophic forgetting.



−1.8 pt vs its own base

72.4 → 70.6 from full-parameter tuning on a narrow domain (Rust pretraining, Verus proofs, C2Rust) — a mild, measurable forgetting.

Still above 4× larger GPT-OSS-120B

70.6 vs 62.0 — general coding competence is preserved, not traded away.

Within ~1.5–2 pt of the leaders

GPT-5-mini and the 122B sibling both score 72.0 on this benchmark.

Takeaways

Three things this 25-minute tour hopefully leaves you with — plus where we go next.

1 **Specialization can beat scale**

A 27B dense model with the right curriculum outperforms 230B–744B open-weight baselines on C2Rust — 87.3% vs 83.9–84.4% — at a fraction of the serving cost.

2 **A curriculum is capability stacking**

Stage 1 buys idiomatic Rust, Stage 2 buys feedback-driven self-repair, Stage 3 buys the task mapping. Each stage mirrors a demand of the evaluation loop.

3 **Grade behavior, not text**

Execution-based verification — compile, run FFI-linked E2E tests, repair within budget — is what makes the 15-point gain meaningful.

FUTURE WORK

Beyond LeetCode → pointer-heavy, real-world C codebases · systematic per-stage ablation · diffusion LLMs as a natural fit for whole-program idiomatic refinement



Shenzhen

Thanks!

Fine-Tuning Qwen3.5-27B for C-to-Rust Code Translation

Pu Zhao, Changdi Yang, Yixiao Chen, Yi Gao, Yifan Cao, Haochen Zeng, Yanzhi Wang — Northeastern University · Aibao LLC

Model · huggingface.co/moxin-org/C2Rust Code · github.com/moxin-org/C2Rust

October 16–17, 2026 Shenzhen, China

